1 Semantics of `verify!`

The `verify!` macro is responsible for checking whether a given proof correctly satisfies the circuit constraints

Listing 1: Macro Definition

```
verify!(circuit_expr, [public_inputs...], proof_expr)
```

The macro expands into:

```
Verifier::verify(&circuit_expr, &[public_inputs...], &proof_expr
    )
```

Listing 2: Verification Function

```
pub fn verify(circuit: &Circuit, public_inputs: &[FieldElement],
     proof: &Proof) -> bool {
    if !Self::check_commitments(&proof.commitments, &proof.
    revealed_witness) {
        return false;
    }
    let mut value_map: HashMap<String, FieldElement> = HashMap::
    new();
    for (name, (val, _blinding)) in &proof.revealed_witness {
        value_map.insert(name.clone(), val.clone());
    }
    for (i, sig) in circuit.inputs.iter().enumerate() {
        if let Some(pub_val) = public_inputs.get(i) {
            if let Signal::Input(ref name) = sig {
                value_map.insert(name.clone(), pub_val.clone());
            }
        }
    }
    if !Self::constraints_satisfied(&circuit.constraints, &mut
    value_map) {
        return false;
    }
    true
}
```

1.1 Background

```
pub struct Circuit {
    pub inputs: Vec<Signal>,
```

```
    pub outputs: Vec<Signal>,
    pub constraints: Vec<Constraint>,
}
```

```
pub struct Proof {
    pub commitments: HashMap<String, Commitment>,
    pub revealed_witness: HashMap<String, (FieldElement,
    FieldElement)>,
}
```

$$\mathcal{V} : C \times I \times P \to \{\text{true}, \text{false}\} \tag{1}$$

where:

$\mathbb{C}$ is a circuit.

$\mathbb{I}$ is a set of public inputs.

$\mathbb{P}$ is a proof (commitments + revealed witness values).

The function outputs `true` if verification succeeds, and `false` otherwise.

2 Semantics

2.1 Extract Circuit Components

Given a circuit C, extract:

$$\begin{aligned} \mathbb{I} &: \text{set of public input signals,} \\ \mathbb{O} &: \text{set of output signals,} \\ \mathbb{C} &: \text{set of constraints.} \end{aligned}$$

Semantics:

$$C = (\mathbb{I}, \mathbb{O}, \mathbb{C}) \tag{2}$$

2.2 Extract Proof Components

Given a proof P, extract:

$$\begin{aligned} \mathbb{K} &: \text{set of commitments,} \\ \mathbb{R} &: \text{set of revealed witness values.} \end{aligned}$$

Semantics:

$$P = (\mathbb{K}, \mathbb{R}) \tag{3}$$

2.3 Check Commitments

Listing 3: Check Commitments

```
fn check_commitments(
    commitments: &HashMap<String, Commitment>,
    revealed_witness: &HashMap<String, (FieldElement,
    FieldElement)>,
) -> bool {
    if commitments.len() != revealed_witness.len() {
        return false;
    }
    for (name, (val, blind)) in revealed_witness.iter() {
        let recomputed = Commitment::new(val, blind);
        if commitments.get(name).map_or(true, |c| recomputed.
    committed_hash != c.committed_hash) {
            return false;
        }
    }
    true
}
....
impl Commitment {
    /// C = SHA256( val_bytes || blind_bytes || prime_bytes )
    pub fn new(val: &FieldElement, blinding: &FieldElement) ->
    Self {
        let val_bytes = val.value.to_le_bytes();
        let blind_bytes = blinding.value.to_le_bytes();

        let mut hasher = Sha256::new();
        hasher.update(&val_bytes);
        hasher.update(&blind_bytes);
        hasher.update(&val.prime.to_le_bytes());

        let result = hasher.finalize();
        Self {
            committed_hash: result.to_vec(),
        }
    }
}
```

For each revealed witness $(s, w_s, r_s) \in \mathbb{R}$, verify:

$$H(w_s, r_s, p) = \mathbb{K}(s) \tag{4}$$

where H is the cryptographic hash function.

2.4 Construct Value Map

Build a value map σ mapping signal names to field elements.

- **Add public inputs:**

$$\sigma(s) = v, \quad \forall s \in \mathbb{I}, v \in \mathbb{I}$$

- **Add revealed witness values:**

$$\sigma(s) = w_s, \quad \forall (s, w_s, r_s) \in \mathbb{R}$$

This results in a mapping:

$$\sigma : S \to \mathbb{F}_p$$

2.5 Evaluate Constraints

Listing 4: Evaluate Constraints

```
fn constraints_satisfied(
    constraints: &[Constraint],
    value_map: &mut HashMap<String, FieldElement>,
) -> bool {
    for constraint in constraints {
        let left_val = Self::eval_signal(&constraint.left,
    value_map).unwrap();
        let right_val = Self::eval_signal(&constraint.right,
    value_map).unwrap();
        let result = match constraint.operation {
            Operation::Add => left_val.add(&right_val),
            Operation::Mul => left_val.mul(&right_val),
            Operation::Sub => left_val.sub(&right_val),
            Operation::Eq => {
                if !left_val.equals(&right_val) {
                    return false;
                }
                left_val.clone()
            }
        };
        match &constraint.output {
            Signal::Output(ref nm) | Signal::Witness(ref nm) =>
    {
                value_map.insert(nm.clone(), result);
            }
            _ => {}
        }
```

```
    }
    true
}
```

For each constraint $(s_1 \oplus s_2 \to s_3) \in \mathbb{C}$:

$$\begin{aligned} v_1 = \sigma(s_1), \quad v_2 = \sigma(s_2), \\ v_3 = v_1 \oplus v_2 \mod p, \\ \sigma(s_3) \stackrel{?}{=} v_3. \end{aligned}$$

If any constraint fails, return `false`.

2.6 Return Verification Result

$$\mathcal{V}(C, I, P) = \begin{cases} \text{true}, & \text{if commitments and constraints hold} \\ \text{false}, & \text{otherwise} \end{cases} \tag{5}$$

3 Using Inference Rules

We define `verify!` as a judgment:

$$C, I, P \Downarrow \text{true} \quad \text{(if verification succeeds)}$$

$$C, I, P \Downarrow \text{false} \quad \text{(if verification fails)}$$

Where:

- $\mathbb{C}$ is the circuit (constraints, inputs, outputs).
- $\mathbb{I}$ is the set of public inputs.
- $\mathbb{P}$ is the proof (commitments + revealed witnesses).
- The output is `true` if verification succeeds, otherwise `false`.

3.1 (1) Commitment Check

$$\frac{\forall (s, w_s, r_s) \in P.\mathbb{R}, \quad H(w_s, r_s, p) = P.\mathbb{K}(s)}{P \Downarrow_{\text{commit}} \text{true}}$$

If all revealed witness values (w_s, r_s) correctly match their commitments $H(w_s, r_s, p)$, the commitment check passes.

Otherwise:

$$\frac{\exists (s, w_s, r_s) \in P.\mathbb{R}, \quad H(w_s, r_s, p) \neq P.\mathbb{K}(s)}{P \Downarrow_{\text{commit}} \text{false}}$$

If any commitment check fails, verification fails immediately.

3.2 (2) Construct Value Map σ

$$\frac{\sigma = \{s \mapsto v \mid s \in \mathbb{I}, v \in I\} \cup \{s \mapsto w_s \mid (s, w_s, r_s) \in P.\mathbb{R}\}}{(C, P) \Downarrow_{\text{map}} \sigma}$$

The value map σ is built by:

- Assigning public input values v to input signals s.
- Assigning revealed witness values w_s to witness signals s.

This map σ will be used for constraint evaluation.

3.3 (3) Constraint Satisfaction

For each constraint $(s_1 \oplus s_2 \to s_3) \in \mathbb{C}$:

$$\frac{\sigma(s_1) = v_1, \quad \sigma(s_2) = v_2, \quad v_3 = v_1 \oplus v_2 \mod p, \quad \sigma(s_3) = v_3}{\mathbb{C}, \sigma \Downarrow_{\text{constraints}} \text{true}}$$

Meaning:

1. Values of s_1 and s_2 are retrieved from σ.
2. The operation $\oplus$ is applied to compute v_3.
3. The computed result v_3 matches $\sigma(s_3)$.

If any constraint fails:

$$\frac{\sigma(s_3) \neq v_3}{\mathbb{C}, \sigma \Downarrow_{\text{constraints}} \text{false}}$$

then verification fails.

3.4 Final Rule: Complete Verification

$$\frac{P \Downarrow_{\text{commit}} \text{true}, \quad (C, P) \Downarrow_{\text{map}} \sigma, \quad \mathbb{C}, \sigma \Downarrow_{\text{constraints}} \text{true}}{C, I, P \Downarrow \text{true}}$$

If all three steps succeed (commitment check, value mapping, and constraint verification), then the proof is valid, and `verify!` returns true.

Otherwise:

$$\frac{P \Downarrow_{\text{commit}} \text{false} \quad \text{or} \quad \mathbb{C}, \sigma \Downarrow_{\text{constraints}} \text{false}}{C, I, P \Downarrow \text{false}}$$

If either:

- A commitment check fails, or
- A constraint evaluation fails,

then verification fails, and `verify!` returns false.